

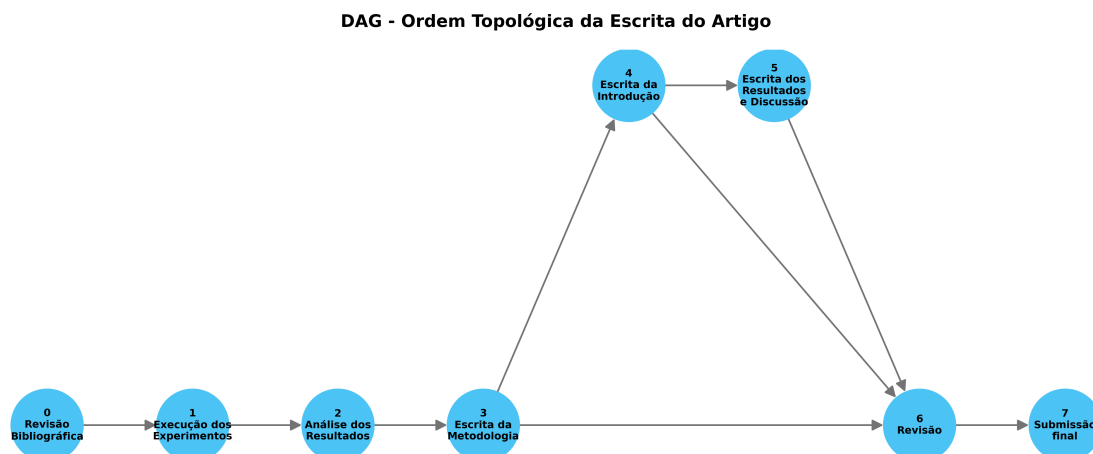
Processamento de Grafos

Discente: Bruno Victor Paiva da Silva

1. Defina um grafo acíclico direcionado relacionado a área do seu mestrado/doutorado e use a sua implementação para ordenar topologicamente esse grafo

Problema: Passos que utilizo para construção de um artigo científico:

- 0 - Revisao Bibliografica;
- 1 - Execucao dos Experimentos;
- 2 - Analise dos Resultados;
- 3 - Escrita da Metodologia;
- 4 - Escrita da Introducao;
- 5 - Escrita dos Resultados e Discussao;
- 6 - Revisao;
- 7 - Submissão final.



```
#include <iostream>
#include <vector>
#include <queue>
#include <string>
```

```

class ArtigoDAG {
private:
    int V;
    std::vector<std::vector<int>> adj;
    std::vector<std::string> nomes;

public:
    ArtigoDAG(int vertices) : V(vertices), adj(vertices), nomes(vertices) {}

    void setNome(int v, std::string nome) {
        nomes[v] = nome;
    }

    void addAresta(int u, int v) {
        adj[u].push_back(v);
    }

    void ordenacaoTopologica() {
        std::vector<int> grauEntrada(V, 0);
        for (int u = 0; u < V; u++) {
            for (int v : adj[u]) {
                grauEntrada[v]++;
            }
        }

        std::priority_queue<int, std::vector<int>, std::greater<int>> q;
        for (int i = 0; i < V; i++) {
            if (grauEntrada[i] == 0) {
                q.push(i);
            }
        }

        std::vector<int> ordem;
        int count = 0;
    }
}

```

```

while (!q.empty()) {
    int u = q.top();
    q.pop();
    ordem.push_back(u);
    count++;

    for (int v : adj[u]) {
        grauEntrada[v]--;
        if (grauEntrada[v] == 0) {
            q.push(v);
        }
    }
}

if (count != V) {
    std::cout << "Erro: Ciclo detectado no grafo.\n";
    return;
}

std::cout << "Ordenacao Topologica do Artigo Academic
o:\n";

for (size_t i = 0; i < ordem.size(); i++) {
    std::cout << i + 1 << ". " << nomes[ordem[i]] <<
"\n";
}
}

};

int main() {
    ArtigoDAG dag(8);

    dag.setNome(0, "Revisao Bibliografica");
    dag.setNome(1, "Execucao dos Experimentos");
    dag.setNome(2, "Analise dos Resultados");
    dag.setNome(3, "Escrita da Metodologia");

```

```

dag.setNome(4, "Escrita da Introducao");
dag.setNome(5, "Escrita dos Resultados e Discussao");
dag.setNome(6, "Revisao");
dag.setNome(7, "Submissão final");

dag.addAresta(0, 1);
dag.addAresta(1, 2);
dag.addAresta(2, 3);
dag.addAresta(3, 4);
dag.addAresta(4, 5);
dag.addAresta(5, 6);
dag.addAresta(4, 6);
dag.addAresta(3, 6);
dag.addAresta(6, 7);

dag.ordenacaoTopologica();

return 0;
}

```

Saída:

```

rm -f artigo_dag
g++ -Wall -Wextra -O3 -std=c++17 -o artigo_dag main.cpp
Ordenacao Topologica do Artigo Academico:
1. Revisao Bibliografica
2. Execucao dos Experimentos
3. Analise dos Resultados
4. Escrita da Metodologia
5. Escrita da Introducao
6. Escrita dos Resultados e Discussao
7. Revisao
8. Submissão final
bruno@pop-os:~/programacao-paralela/algoritmos-avancados/grafos/dag$

```

2. Implemente e aponte vantagens e desvantagens dos algoritmos de Kruskal e Prim para gerar uma árvore de abrangência mínima. Use exemplos práticos da sua área para demonstrar suas conclusões.

Cenário 1: Desempenho em Grafos Esparsos:

O primeiro cenário de testes avalia os algoritmos em um grafo esparsos, estruturado para simular a topologia física de um *cluster* computacional (10 vértices e 17 arestas)

Cenário 2: Desempenho em Grafos Densos (Matriz Lógica e Latência)

O segundo cenário modela um grafo denso, representando uma simulação matemática de uma malha lógica totalmente interconectada, com 500 vértices e 124.750 arestas de latência simulada.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <queue>
#include <chrono>
#include <random>
#include <iomanip>

struct Aresta {
    int u, v;
    double peso;
    bool operator<(const Aresta& outra) const {
        return peso < outra.peso;
    }
};

class DSU {
    std::vector<int> pai, rank;
public:
    DSU(int n) {
        pai.resize(n);
        rank.resize(n, 0);
        for (int i = 0; i < n; i++) pai[i] = i;
    }
    int encontrar(int i) {
        if (pai[i] == i) return i;
        return pai[i] = encontrar(pai[i]);
    }
};
```

```

    }
    void unir(int i, int j) {
        int raiz_i = encontrar(i);
        int raiz_j = encontrar(j);
        if (raiz_i != raiz_j) {
            if (rank[raiz_i] < rank[raiz_j]) pai[raiz_i] = ra
            iz_j;
            else if (rank[raiz_i] > rank[raiz_j]) pai[raiz_j]
            = raiz_i;
            else {
                pai[raiz_j] = raiz_i;
                rank[raiz_i]++;
            }
        }
    }
};

class MSTBenchmark {
private:
    int V;
    std::vector<Aresta> arestas;
    std::vector<std::vector<std::pair<int, double>>> adj;

public:
    MSTBenchmark(int vertices) : V(vertices), adj(vertices)
    {}

    void addAresta(int u, int v, double peso) {
        arestas.push_back({u, v, peso});
        adj[u].push_back({v, peso});
        adj[v].push_back({u, peso});
    }

    void kruskal() {
        auto start = std::chrono::high_resolution_clock::now
        ();

```

```

double custo_mst = 0;
int arestas_mst = 0;
DSU dsu(V);

std::vector<Aresta> arestas_ordenadas = arestas;
std::sort(arestas_ordenadas.begin(), arestas_ordenada
s.end());

for (const auto& aresta : arestas_ordenadas) {
    if (dsu.encontrar(aresta.u) != dsu.encontrar(aresta.v)) {
        dsu.unir(aresta.u, aresta.v);
        custo_mst += aresta.peso;
        arestas_mst++;
        if (arestas_mst == V - 1) break;
    }
}

auto end = std::chrono::high_resolution_clock::now();
std::chrono::duration<double, std::milli> tempo = end
- start;

std::cout << "[Kruskal]\n";
std::cout << "    Custo Total: " << custo_mst << "\n";
std::cout << "    Tempo: " << tempo.count() << " ms\n";
}

void prim() {
    auto start = std::chrono::high_resolution_clock::now
();

    double custo_mst = 0;
    std::vector<bool> na_mst(V, false);
    int arestas_mst = 0;

```

```

using pdi = std::pair<double, int>;
std::priority_queue<pdi, std::vector<pdi>, std::greater<pdi>> pq;

pq.push({0.0, 0});

while (!pq.empty() && arestas_mst < V) {
    double peso = pq.top().first;
    int u = pq.top().second;
    pq.pop();

    if (na_mst[u]) continue;

    na_mst[u] = true;
    custo_mst += peso;
    arestas_mst++;

    for (const auto& vizinho : adj[u]) {
        int v = vizinho.first;
        double peso_vizinho = vizinho.second;
        if (!na_mst[v]) {
            pq.push({peso_vizinho, v});
        }
    }
}

auto end = std::chrono::high_resolution_clock::now();
std::chrono::duration<double, std::milli> tempo = end
- start;

std::cout << "[Prim com Binary Heap]\n";
std::cout << "    Custo Total: " << custo_mst << "\n";
std::cout << "    Tempo: " << tempo.count() << " ms\n";
}
};

```



```

void cenarioEsperso() {
    std::cout << "--- Questao 1: Grafo Esperso (Rede de Clust
er Fisica) ---\n";
    std::cout << "Vertices (V): 10, Arestas (E): 17\n";

    MSTBenchmark cluster(10);

    for(int i = 0; i < 8; i++) cluster.addAresta(i, 8, 1.0);
    cluster.addAresta(8, 9, 2.0);
    for(int i = 0; i < 8; i++) cluster.addAresta(i, 9, 3.0);

    cluster.kruskal();
    cluster.prim();
}

void cenarioDenso() {
    int V = 500;
    int E = (V * (V - 1)) / 2;
    std::cout << "\n--- Questao 2: Grafo Denso (Simulacao de
Matriz Logica) ---\n";
    std::cout << "Vertices (V): " << V << ", Arestas (E): " <
< E << "\n";

    MSTBenchmark grafo_denso(V);

    std::mt19937 rng(42);
    std::uniform_real_distribution<double> dist(0.5, 10.0);

    for (int i = 0; i < V; i++) {
        for (int j = i + 1; j < V; j++) {
            grafo_denso.addAresta(i, j, dist(rng));
        }
    }

    grafo_denso.kruskal();
    grafo_denso.prim();
}

```

```

}

int main() {
    std::cout << std::fixed << std::setprecision(4);
    cenarioEsperso();
    cenarioDenso();
    return 0;
}

```

Saída:

```

--- Questao 1: Grafo Esperso (Rede de Cluster Fisica) ---
Vertices (V): 10, Arestas (E): 17
[Kruskal]
    Custo Total: 10.0000
    Tempo: 0.0053 ms
[Prim com Binary Heap]
    Custo Total: 10.0000
    Tempo: 0.0066 ms

--- Questao 2: Grafo Denso (Simulacao de Matriz Logica) ---
Vertices (V): 500, Arestas (E): 124750
[Kruskal]
    Custo Total: 261.1276
    Tempo: 11.1327 ms
[Prim com Binary Heap]
    Custo Total: 261.1276
    Tempo: 8.0299 ms
bruno@pop-os:~/programacao-paralela/algoritmos-avancados/grafos/kruskal$

```

3. Implemente o algoritmo do Dijkstra e utilize-o para resolver um problema prático da sua área de interesse

Problema: O algoritmo implementado resolve o problema de otimização de rotas e cálculo de caminho mínimo em grafos ponderados com pesos não-negativos.

```

#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <limits>

```

```

#include <algorithm>

const int INF = std::numeric_limits<int>::max();

struct Aresta {
    int destino;
    int tempo_minutos;
};

class MapaGPS {
private:
    int V;
    std::vector<std::vector<Aresta>> adj;
    std::vector<std::string> locais;

public:
    MapaGPS(int vertices) : V(vertices), adj(vertices), locais(vertices) {}

    void setNomeLocal(int id, std::string nome) {
        locais[id] = nome;
    }

    void addRua(int u, int v, int tempo) {
        adj[u].push_back({v, tempo});
        adj[v].push_back({u, tempo});
    }

    void dijkstra(int origem, int destino) {
        std::vector<int> dist(V, INF);
        std::vector<int> pai(V, -1);

        using pii = std::pair<int, int>;
        std::priority_queue<pii, std::vector<pii>, std::greater<pii>> pq;
    }
};

```

```

    dist[origem] = 0;
    pq.push({0, origem});

    while (!pq.empty()) {
        int tempo_atual = pq.top().first;
        int u = pq.top().second;
        pq.pop();

        if (tempo_atual > dist[u]) continue;

        for (const auto& vizinho : adj[u]) {
            int v = vizinho.destino;
            int peso = vizinho.tempo_minutos;

            if (dist[u] + peso < dist[v]) {
                dist[v] = dist[u] + peso;
                pai[v] = u;
                pq.push({dist[v], v});
            }
        }
    }

    imprimirRota(origem, destino, dist, pai);
}

private:
    void imprimirRota(int origem, int destino, const std::vector<int>& dist, const std::vector<int>& pai) {
        if (dist[destino] == INF) {
            std::cout << "Rota inacessivel!\n";
            return;
        }

        std::vector<int> caminho;
        for (int v = destino; v != -1; v = pai[v]) {
            caminho.push_back(v);
        }
    }

```

```

    }
    std::reverse(caminho.begin(), caminho.end());

    std::cout << "Menor latencia total: " << dist[destino] << " ms\n";
    std::cout << "Caminho a seguir:\n";
    for (size_t i = 0; i < caminho.size(); i++) {
        std::cout << locais[caminho[i]];
        if (i < caminho.size() - 1) std::cout << " -> ";
    }
    std::cout << "\n\n";

    std::cout << " Distancias de '" << locais[origem] <<
    "' para todos os outros nos \n";
    for (int i = 0; i < V; i++) {
        std::cout << locais[origem] << " -> " << locais
[i] << ": ";
        if (dist[i] == INF) {
            std::cout << "Inacessivel\n";
        } else {
            std::cout << dist[i] << " ms\n";
        }
    }
}

};

int main() {
    MapaGPS mapa(6);

    mapa.setNomeLocal(0, "A");
    mapa.setNomeLocal(1, "B");
    mapa.setNomeLocal(2, "C");
    mapa.setNomeLocal(3, "D");
    mapa.setNomeLocal(4, "E");
    mapa.setNomeLocal(5, "F");

```

```

    mapa.addRua(0, 1, 5);
    mapa.addRua(0, 2, 2);
    mapa.addRua(1, 3, 4);
    mapa.addRua(1, 4, 2);
    mapa.addRua(2, 1, 8);
    mapa.addRua(2, 4, 7);
    mapa.addRua(3, 4, 6);
    mapa.addRua(3, 5, 3);
    mapa.addRua(4, 5, 1);

    mapa.dijkstra(0, 5);

    return 0;
}

```

```

rm -f dijkstra_gps
g++ -Wall -Wextra -O3 -std=c++17 -o dijkstra_gps main.cpp
Menor latencia total: 8 ms
Caminho a seguir:
A -> B -> E -> F

Distancias de 'A' para todos os outros nos
A -> A: 0 ms
A -> B: 5 ms
A -> C: 2 ms
A -> D: 9 ms
A -> E: 7 ms
A -> F: 8 ms
❖ bruno@pop-os:~/programacao-paralela/algoritmos-avancados/grafos/dijkstra$

```